

Fundamentos de Aprendizaje Automático 2024/2025

PRÁCTICA Nº 1: Naive Bayes, K-NN y Scikit Learn

1. Objetivo

El objetivo de esta práctica es implementar los algoritmos de clasificación *Naive-Bayes* y *Vecinos Próximos* (K-NN). El algoritmo *Naive-Bayes* clasifica cada ejemplo en la clase cuya probabilidad a posteriori es máxima, suponiendo que todos los atributos con los que se representan las instancias son independientes. Por su parte, el algoritmo *K-NN* es un método que trata de clasificar cada ejemplo en función de la clase mayoritaria de los vecinos a ese ejemplo, considerando como vecino todo aquel que está a una determinada distancia, definida en base a una métrica, como puede ser la distancia Euclídea.

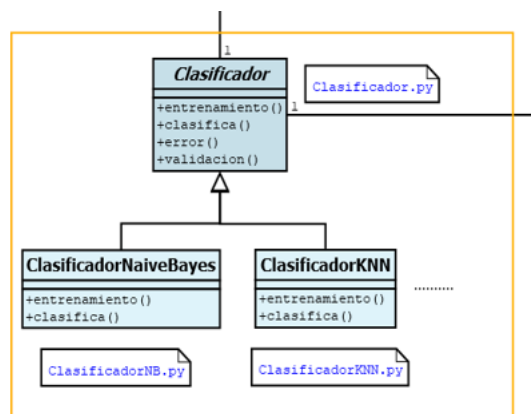
Tras nuestra implementación utilizaremos las librerías proporcionadas por Scikit Learn para Naive Bayes y K-NN, de manera que podamos comparar los resultados de ambas alternativas.

2. Conjuntos de datos

Para probar los algoritmos Naive Bayes y Vecinos Próximos utilizaremos los conjuntos de datos *heart* y *wdbc* proporcionados en Moodle:

3. Estructura de clasificadores

Como se comentó en la introducción, los clasificadores se implementarán en base a una jerarquía cuya superclase es la clase *Clasificador*. En la figura se resume lo explicado en la práctica inicial:



La clase *Clasificador* tiene como mínimo cuatro métodos: dos propios y dos abstractos. Los métodos abstractos son **entrenamiento** y **clasifica**. Como es lógico, su



implementación depende del algoritmo de clasificación considerado, por lo que deben implementarse en las subclases. Los métodos **error** y **validación** son comunes. El primero es auto explicativo, mientras que el segundo actúa como una especie de Main de clasificación. En Moodle puede encontrarse la plantilla de esta superclase con indicaciones sobre parámetros de entrada, retorno y pasos principales a desarrollar.

4. Estandarización de datos

Es posible que sea necesario realizar una estandarización de datos para ajustar las escalas de los atributos numéricos y evitar así sesgos no deseados por la diferencia de escala en algunos algoritmos de aprendizaje. Al objeto de ampliar nuestra aplicación con esta funcionalidad, se implementará un método nuevo en la clase que consideres más apropiada, justificando la elección:

- `estandarizarDatos(media=True, std=True)`: este método estandariza cada uno de los atributos numéricos en la matriz `datos` utilizando la media del atributo si `media=True`, o cero si `media=False` y la desviación típica del atributo si `std=True`, o uno si `std=False`. Es decir, su comportamiento será el mismo al del método correspondiente de la librería de Scikit-learn que se comentará en el apartado 5.4.

5. Scikit-learn

5.1 Naive Bayes

Scikit-learn proporciona diferentes implementaciones para el algoritmo Naive Bayes (http://scikit-learn.org/stable/modules/naive_bayes.html). En esta práctica, consideraremos las siguientes funciones:

- **MultinomialNB**: es la versión para datos distribuidos multinomialmente. El valor a priori de cada clase se puede asumir uniforme (`fit_prior=False`) o se puede obtener a partir de los datos (`fit_prior=True`). Esta función también permite especificar un parámetro aditivo de suavizado `alpha`. `alpha=0` no realiza ningún tipo de suavizado y `alpha=1` implementa la corrección de Laplace. Este tipo de distribución se utiliza clásicamente en problemas de análisis de texto, como la clasificación de correo en spam/no spam, ya que la distribución multinomial no representa categorías, sino cuentas, por ejemplo, de palabras.
- **GaussianNB**: la verosimilitud de las variables se asume que sigue una distribución normal.
- **CategoricalNB**: es la versión Naive Bayes para datos categóricos. Asume que cada característica tiene su propia distribución categórica. Como **MultinomialNB**, permite indicar los argumentos `fit_prior` y `Alpha`, así como el número mínimo de categorías por característica: `min_categories`.



5.2 K-NN

Scikit-learn implementa dos clasificadores basados en vecindad. Para los propósitos de esta práctica interesa **KNeighborsClassifier**. Un argumento de este método es **metric**, que permite indicar la distancia a utilizar.

5.3 Particionado

En cuanto al particionado de los datos, el submódulo **model_selection** de Scikit-learn implementa diferentes estrategias (http://scikit-learn.org/stable/model_selection.html). En lo que se refiere a las estrategias de validación simple y validación cruzada a implementar en esta práctica, estas son algunas de las funciones más relevantes:

- **Validación simple:** La función **train_test_split** hace transparente la gestión de índices ya que devuelve directamente los conjuntos de entrenamiento y validación para un conjunto de datos dado. Por su parte, la instanciación de la clase **ShuffleSplit** proporciona los índices para dividir los datos en subconjuntos de training y test. Los parámetros de estas funciones/constructores permiten indicar la proporción de patrones a utilizar en test o el número de particiones a generar, entre otros.
- **Validación cruzada:** Las funciones **cross_val_score** y **cross_val_predict** devuelven directamente los errores por validación cruzada por cada *fold* y las predicciones por cada patrón, respectivamente. Por su parte, la instanciación de la clase **KFold** genera los índices de las particiones, igual que hace la Estrategia de Particionado implementada en la práctica de introducción. Entre otras posibilidades, los parámetros de estas funciones/constructores permiten indicar el número de *folds* a generar o imponer la permutación aleatoria de los datos antes de generar la partición.

5.4 Estandarización de datos

La librería Scikit-learn proporciona el método **StandardScaler** (<https://scikit-learn.org/dev/modules/generated/sklearn.preprocessing.StandardScaler.html>) para realizar la estandarización de los datos, es decir, la conversión de los atributos de acuerdo a una distribución de media cero y desviación estándar uno.

6. Actividades

La planificación temporal sugerida y las actividades a llevar a cabo son las siguientes:

- **1ª semana:** Implementar el clasificador *Naive-Bayes* y aplicar este algoritmo a los conjuntos de datos **wdbc** y **heart**, para obtener los porcentajes de error en clasificación en cada caso, tanto con validación cruzada como con validación simple. En todos los casos se deben hacer los cálculos sin aplicar y aplicando la



corrección de Laplace. Tened en cuenta que ambos conjuntos de datos incluyen atributos continuos, por lo que en ese caso supondremos que estos atributos siguen una distribución normal dada la clase. Para ello se debe calcular la media y la varianza de cada atributo continuo. Pueden ser de utilidad las funciones para la distribución normal del módulo `scipy.stats.norm`.

- *2ª semana*: Comparar los resultados obtenidos en nuestra implementación con los que proporciona la librería de aprendizaje automático Scikit-learn (<http://scikit-learn.org/stable/>). Reproducir el diseño experimental utilizado anteriormente para la validación simple y la validación cruzada. Evaluar y comentar las hipótesis obtenidas con el clasificador Naive-Bayes según los criterios que se definen en el apartado 7 destinado a la entrega.
- *3ª semana*: Implementar el algoritmo de *vecinos próximos* (clase **ClasificadorKNN**) para realizar una tarea de clasificación de los conjuntos de datos *heart* y *wdbc*, probando con diferentes valores de vecindad ($K=1, 5, 11$ y 21). En el uso del algoritmo de vecinos próximos es recomendable normalizar los atributos. La normalización es importante puesto que evita problemas de escala. Para llevar a cabo la normalización se añadirán nuevos métodos, tal y como se describe en el apartado 4. Aparte de la normalización y el valor de K , un aspecto importante en vecinos próximos es la métrica o distancia utilizada. En la implementación propia se debe trabajar con la distancia Euclídea.
- *4ª semana*: Analizar los conjuntos de datos *heart* y *wdbc* con la librería de Scikit-learn para vecinos próximos, utilizando los mismos valores de K y con la misma métrica Euclídea, tanto en validación simple como en validación cruzada. Comparar con los resultados obtenidos con la implementación propia, tanto en datos normalizados como sin normalizar.

7. Fecha de entrega y entregables

31 de Octubre de 2024. La entrega debe realizarse antes del comienzo de la clase de prácticas correspondiente. Se deberá entregar un fichero comprimido **.zip** con nombre **FAAP1_<grupo>_<pareja>.zip** (ejemplo **FAAP1_1461_1.zip**) y el siguiente contenido:

1. **Notebook (.ipynb)**. El Notebook debe estructurarse para contener los siguientes apartados:

Apartado 1	Naive-Bayes propio
	• Tabla con los resultados de la ejecución para los conjuntos de datos analizados (<i>wdbc</i> y <i>heart</i>). Considerar los dos tipos de particionado. Los resultados se refieren a las tasas de error y deben mostrarse tanto con la corrección de Laplace como sin ella. Se debe incluir tanto el promedio de error como su desviación típica. <u>Es importante mostrar todos los resultados</u>

	agrupados en una tabla para facilitar su evaluación. Breve análisis de los resultados anteriores. Discutir el efecto Laplace
Apartado 2	Naive-Bayes Scikit-Learn - Tabla de resultados equivalente a la anterior, pero utilizando los métodos del paquete scikit-learn: MultinomialNB, GaussianNB y CategoricalNB. ¿Existe algún problema con alguno de estos métodos en alguno de los dos ficheros? En caso afirmativo, ¿en cuál? ¿por qué? ¿cómo podría resolverse?
Apartado 3	K-NN propio - Resultados en forma de tabla de la clasificación mediante vecinos próximos para los diferentes valores de vecindad en los conjuntos de datos propuestos. Obtener los resultados tanto para datos estandarizados como sin estandarizar, con el objetivo de justificar el rendimiento del algoritmo en base a estas características. Separar por tipo de validación (simple, cruzada). - Análisis de resultados, del impacto de K y de la normalización
Apartado 4	K-NN Scikit-Learn - Tabla de resultados equivalente a la del apartado anterior para las ejecuciones realizadas con la librería KNeighborsClassifier en los mismos valores de K, datos normalizados y no normalizados. - Análisis de resultados, del impacto de K y de la normalización
Apartado 5	Conclusión Comparar y analizar a modo de resumen los resultados propios con los de Scikit-Learn para los dos algoritmos de aprendizaje y ambos conjuntos. Utiliza una tabla para comparar los resultados

2. Notebook exportado como fichero html.

3. Código Python (**ficheros.py**) necesario para la correcta ejecución del Notebook. Esto incluye todos los ficheros implementados hasta ahora: Datos, EstrategiaParticionado, Clasificador, ClasificadorNB y ClasificadorKNN. Para la parte de Scikit Learn puedes optar por incluir un fichero .py separado con esta parte, o incluir los métodos de Scikit Learn como subclases en el diseño de la arquitectura propia.

8. Puntuación de cada apartado



La valoración de cada apartado será la siguiente:

- ✓ **Naive-Bayes propio:** 2,5 puntos.
- ✓ **Naive-Bayes con Scikit-Learn:** 1,5 puntos.
- ✓ **K-NN propio:** 2,5 puntos
- ✓ **K-NN con Scikit-Learn:** 1,5 puntos
- ✓ **Presentación y discusión de los resultados:** 2 puntos.